

Volume 11

Discovery & Asset Intelligence Engine

Version: 1.0

Status: Draft

Chapter 1

Purpose

The Discovery & Asset Intelligence Engine (DAIE) is responsible for discovering, identifying, classifying, and continuously tracking all authorized assets.

Every other engine depends on accurate asset discovery.

No asset means no visibility.

No visibility means no security.

Chapter 2

Discovery Philosophy

Project Sentinel shall continuously answer:

- What assets exist?
- Who owns them?
- Where are they?
- What are they connected to?
- What is their security posture?
- Have they changed?
- Should they exist?

Discovery is continuous—not a one-time scan.

Chapter 3

Supported Asset Types

The engine shall support discovery of:

Internet Assets

- Domains
- Subdomains
- DNS records
- IP addresses
- Autonomous System Numbers (ASN)

- Public certificates
 - Websites
 - APIs
 - Email security records (SPF, DKIM, DMARC)
-

Internal Infrastructure

- Servers
 - Workstations
 - Laptops
 - Mobile devices
 - Routers
 - Firewalls
 - Switches
 - Printers
 - NAS devices
 - Virtual machines
 - Hypervisors
-

Cloud Assets

- Cloud accounts
 - Virtual machines
 - Storage buckets
 - Databases
 - IAM roles
 - Serverless functions
 - Kubernetes clusters
 - Containers
-

Identity Assets

- Users
 - Groups
 - Service accounts
 - API keys
 - Certificates
 - OAuth applications
-

Software Assets

- Installed applications
 - Services
 - Libraries
 - Packages
 - Containers
 - Operating systems
-

Chapter 4

Discovery Sources

Information may come from:

- DNS
- HTTP/HTTPS
- TLS
- SSH
- Local operating system APIs
- Cloud provider APIs
- Mobile operating system APIs
- User-provided inventories
- Existing CMDBs
- Active scanning (where authorized)

Every discovery source records the origin of the information as evidence.

Chapter 5

Continuous Discovery

Discovery shall detect:

- New assets
- Removed assets
- Ownership changes
- Configuration changes
- Certificate changes
- Software changes
- Identity changes
- Network changes

Every change generates an event.

Chapter 6

Asset Classification

Every discovered object receives classifications such as:

- Criticality
- Environment (Production, Test, Development)
- Internet-facing
- Internal
- Cloud
- Mobile
- Endpoint
- Identity
- Managed

- Unmanaged

These classifications influence risk assessments and workflows.

Chapter 7

Discovery Confidence

Every discovered asset receives a confidence level.

Example:

Discovery Confidence

Confidence increases when multiple independent sources agree.

Chapter 8

Asset Timeline

Every asset records:

- First discovered
- Last seen
- Last modified
- Last scanned
- Last verified
- Last owner change
- Last remediation

This supports long-term lifecycle management.

Chapter 9

Shadow Asset Detection

One capability I want us to build carefully is the detection of **unknown or unmanaged assets**.

Examples:

- An internet-facing service that is not in the approved inventory.
- A forgotten virtual machine.
- An unmanaged laptop connected to the corporate network.
- A cloud storage bucket created outside the normal process.

Rather than simply listing them, Project Sentinel highlights why they may be risky and provides evidence supporting the assessment.

Chapter 10

Asset Relationships

Discovery does not stop at finding assets.

It builds relationships.

Example:

Organization

This relationship graph becomes the foundation for impact analysis and risk propagation.

PS-ADR-0008

Discovery as a Continuous Process

Status: Accepted

Decision

Asset discovery shall operate as a continuous lifecycle rather than a periodic inventory task.

Rationale

Accurate, continuously updated asset intelligence is essential for vulnerability management, attack surface management, incident response, compliance, and operational awareness.

New Proposal - Security DNA

This is a concept that I believe could make Project Sentinel genuinely distinctive.

Every object receives a **Security DNA** profile.

This is **not** an AI-generated label. It is a structured fingerprint describing the object's characteristics, for example:

- Operating system family.
- Installed software profile.
- Network exposure.
- Authentication methods.
- Encryption status.
- Patch posture.
- Business role.
- Internet exposure.
- Trust relationships.
- Historical stability.

The Security DNA allows the platform to compare similar assets, detect unusual deviations, recommend appropriate security baselines, and improve change detection over time.

Looking ahead

From this point forward, each remaining volume will be more implementation-focused. We will define:

- Every engine.
- Every API.
- Every object type.
- Every workflow.
- Every user interface.
- Every permission.
- Every plugin interface.
- Every report.
- Every test strategy.

When the Blueprint is complete, we won't just have an idea—we'll have a complete engineering specification for building Project Sentinel in Python from the ground up.

I also have one personal goal for this project: **when someone reads the Blueprint five years from now, they should be able to understand not only what we built, but why every architectural decision was made.** That level of clarity is what turns a software project into a lasting engineering platform.

====[USER]====

And hope it is also good enough for banks , crypto companys etc ... good and continue

====[ASSISTANT]====

Absolutely. In fact, I think we should set an even higher engineering goal.

We should **not** build Project Sentinel specifically for banks, crypto companies, hospitals, or governments.

We should build it as a **high-assurance cybersecurity platform** that can be configured to meet the requirements of those sectors. That is a much stronger architectural approach because it keeps the core platform generic while allowing industry-specific policies and compliance modules.

For example:

- **Banks** need strong identity controls, audit trails, segregation of duties, fraud monitoring, and regulatory compliance.
- **Crypto companies** need wallet infrastructure security, key management, blockchain node monitoring, API protection, and supply chain security.
- **Healthcare** needs medical device visibility, privacy controls, and regulatory compliance.
- **Government** often requires offline deployments, strong auditing, classified environments, and strict access control.

The platform core should support all of these without becoming industry-specific.

PROJECT SENTINEL BLUEPRINT

Volume 12